Proyecto- Etapa 1 – Semantic Web (MISIS)

Descripción General

Para la etapa numero 1 del proyecto, se trabajo con el archivo csv **"grupo_1"** el cual contiene 100.000 registros de canciones. Cada registro tenia un identificador de la canción, titulo, nombre del artista, lanzamiento, año, duración e identificador del artista en MusicBrainz.

La fuente externa utilizada para esta etapa fue MusicBrainz, una base de datos musical que se podía consultar a través de una API. Esta fuente fue apropiada porque el archivo original ya incluía el identificador **"artista_mbid", lo** cual permitió consultar información adicional de los artistas y relacionarla directamente con las canciones.

Por cada artista se consultaron los siguientes datos:

- Identificador del artista en la base de datos.
- Nombre registrado.
- Tipo de artista (Persona o Grupo).
- País o Región.
- Fecha de nacimiento (Cuando el artista es una persona).
- Fecha de formación (Cuando el artista es un grupo).
- Géneros musicales asociados.
- Nombre y URL de la fuente consultada.

Proceso de Extracción

Para el proceso de extracción se utilizo Python en Visual Studio Code y se crearon 3 archivos con sus funcionalidades.

1. **"analizar_data.py"**: Este archivo revisa revisa el CSV original y muestra la cantidad de filas, canciones, títulos y artistas únicos, canciones con y sin año y posibles duplicados.
2. **"consultar_musicbrainz.py":** En este archivo se obtiene los artistas sin repetir su identificador y consulta su información en la API de MusicBrainz. Los resultados exitosos se guardan en **artistas_final.csv** y los errores encontrados durante las consultas se registran en **"errores_producidos.csv".**
3. **"generación_datos.py":** Aquí se realiza las relaciones de las canciones del archivo original con la información de los artistas consultados y genera el archivo **"canciones_final.csv".**

La forma en que se realiza la relación es comparando el **"artist_mbid"** del archivo original con el **"mbid_artista"** del archivo artistas. De esta manera, la información externa queda conectada con los registros originalmente entregados.

El código de consulta revisa cuales artistas ya están guardados antes de mirar MusicBrainz. Si el artista esta guardado, pues no vuelve a agregarlo y esto me permite continuar con el proceso sin repetir consultas exitosas. A su vez, para disminuir los errores, hay un intervalo de tiempo antes de mandar la siguiente consulta y si falla una consulta con errores 429 o 503, vuelve a intentar para que no tenga que estar yo siempre mandando el comando para que vuelva a repetirse la consulta.

Hablando de comandos, para ejecutar el proceso de consulta se abre una terminal en Visual Studio Code y confirmamos que estemos dentro del proyecto. Primero mandamos comando **"Python Src/analizar_data.py",** que analiza el archivo original y muestra sus cantidades. Luego sigue **"Python Src/consultar_musicbrainz.py –limite numero"** donde el numero indica cuantos artistas queremos procesar. Se omiten los artistas ya guardados y consulta los que nos faltan. Si se presentan errores temporales, el comando se puede ejecutar nuevamente sin perder las consultas exitosas. Finalmente, se ejecuta **"Python Src/generación_datos.py"** que hace las conexiones de las canciones con sus artistas mediante el identificador **"artista_mbid"** y genera un archivo final.

Resultados

El archivo consolidado conserva las 100.000 canciones del archivo original, por lo que contiene más de 75.000 instancias. De estas canciones, una parte pequeña ha sido enriquecida con información de MusicBrainz hasta el momento.

Hasta el momento, se realizo una consulta de 1000 artistas y se obtuvieron los siguientes datos:

- 1000 artistas consultados
- 992 consultas exitosas
- 8 consultas fallidas (errores).
- 100.000 canciones finales.
  5.974 canciones enriquecidas.
- 94.026 canciones sin enriquecer.

Retos y Problemas encontrados

Uno de los principales retos se presentó durante las consultas a MusicBrainz. Inicialmente, el programa no contaba con un mecanismo para conservar los resultados anteriores. Por esta razón, cada nueva ejecución reemplazaba la información que ya se había obtenido. Por ejemplo, si se consultaban 200 artistas y luego se intentaba aumentar el límite, los resultados anteriores podían perderse. Para solucionar este problema, el programa guarda las consultas exitosas en artistas_final.csv. Antes de consultar un artista, revisa si ya está guardado. Así no repite las consultas anteriores y solo busca los artistas que faltan.

Otro reto fue el tiempo necesario para realizar las consultas. En una de las pruebas pasadas, consultar 200 artistas tomo entre 30 a 35 minutos. Si hubiera aumentado la cantidad de artistas, el proceso pudo haber tardado varias horas debido esperas en las solicitudes o los mismos límites establecidos por la Api.

También se presentaron errores tipo 429 y 504 cuando el servicio estaba ocupado o se alcanzaba el límite de solicitudes. La cantidad de errores que se presentaban eran particularmente un 20% de las consultas realizadas. Para disminuir estos errores, se agrego un intervalo entre consultas y un mecanismo de reintentos automático.

Finalmente, en el proceso de consultas no todas las canciones tienen un **“artist_mbid”.** Estas canciones no pueden relacionarse directamente con MusicBrainz con el método utilizado.